



Application Report

Application	HideU: Calculator Lock
Package name	com.calculator.hideu
Version	2.3.8
SHA-256 hash	7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8
Test date	2026-05-04 21:43:37.165315
Category	Tools

Risk Scores per Category

PRIVACY ■■■	FINANCIAL ■■■	SYSTEM ■■■	RELIABILITY ■■■
-----------------------	-------------------------	----------------------	---------------------------

Privacy tests:	2, 12, 34, 37, 47, 72, 80, 119, 180, 270, 271, 272, 273, 199, 235, 262, 263, 266, 267,
Financial tests:	96, 119, 274, 210, 212, 235, 243, 244, 245, 246, 247, 248, 249, 252, 255, 257, 264, 265,
System tests:	96, 98, 99, 100, 142, 159, 165, 190, 191, 196, 210, 212, 230, 231, 234, 258, 264, 265,
Reliability tests:	3, 90, 91, 165, 166, 187, 221, 222, 223, 226, 227, 228, 268,

Zou je deze app gebruiken?

Bepaal per categorie (zie de lijst hierboven van tests per categorie) de som van test outcome(s) **1.000**

Welke aspect scoort het slechts? Welke test(s) is(zijn) daar verantwoordelijk voor?

Zou je deze app gebruiken? Waarom wel of waarom niet?

All scores

Test Results

2 Testing Local Storage for Sensitive Data

This test verifies whether the application writes sensitive data to external storage without encryption.

Apps that request dangerous external storage permissions and use external storage APIs such as `getExternalStorageDirectory` may expose sensitive data to other applications or users with physical access to the device. The test fails only when all three conditions are met: a dangerous storage permission is declared, an external storage API is used, and sensitive data is written without encryption in the same file.

The test passes if the app uses secure internal storage, applies encryption before writing to external storage, or does not write sensitive data to external paths.

Test result: **1.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.021**

Evidence:

Permission android.permission.WRITE_EXTERNAL_STORAGE found in manifest
Permission android.permission.READ_EXTERNAL_STORAGE found in manifest
Permission android.permission.MANAGE_EXTERNAL_STORAGE found in manifest
Found external storage API usage
Found sensitive unencrypted write at least in `GsonRequestBodyConverter.java`

3 Testing the Purposes of Keys

This test identifies the use of weak or insecure cryptographic algorithms in the application source code.

Weak algorithms such as DES, Triple DES (DESede), RC2, RC4, Blowfish, MD4, and ECB mode are detected by scanning Java source files for `Cipher.getInstance()` calls with vulnerable parameters. Strong algorithms such as AES/GCM and RSA/OAEP are not flagged. MD5 and SHA-1 are also excluded because they have legitimate non-encryption uses such as checksums and fingerprints.

The test fails if any weak algorithm usage is found in the decompiled source code.

Test result: **1.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.021**

Evidence:

Scanning for weak cryptographic algorithms...
ECB: 1 usage(s) found
Example: `com/inmobi/media/AbstractC10066p4.java:46`
Code: `Cipher r52 = Cipher.getInstance("AES/ECB/PKCS7Padding"); // Catch: Throwable...`

12 Testing Random Number Generation

This test identifies the use of non-cryptographically secure random number generators in combination with sensitive data handling.

Insecure random number generators such as `java.util.Random`, `Math.random()`, `kotlin.random.Random`, and `ThreadLocalRandom` are not suitable for security-sensitive operations such as generating passwords, tokens, or session identifiers. The test scans Java and Smali files for these patterns when they appear near sensitive keywords such as `password`, `token`, or `session`.

The test fails if insecure randomness is detected in a context involving sensitive data handling.

Test result: **1.000** Test weight: **MEDIUM RISK** Weight value: **0.062** Test outcome: **0.062**

Evidence:

Insecure random algorithm found, at least in: `{('tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes10/com/mbridge/msdk/setting/util/b.smali', 'Math.random()')}`

34 Determining Whether the Keyboard Cache Is Disabled for Text Input Fields

This test checks whether input fields in the application's layout files are configured with `android:inputType` including the `textNoSuggestions` flag.

When `android:inputType` does not include `textNoSuggestions`, the Android keyboard may cache user input and present it as autocomplete suggestions in future sessions, potentially exposing sensitive data that was entered in those fields. This is particularly relevant for fields handling usernames, passwords, and other private information.

The test fails if any layout file contains an `EditText` with `android:inputType` set to a value that does not include `textNoSuggestions`.

Test result: **1.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.021**

Evidence:

```
Found android:inputType without value textNoSuggestions in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/res/layout-land/
dialog_color_picker.xml
```

37 Determining Whether the Keyboard Cache Is Disabled for Text Input Fields

This test verifies that the application's minimum supported SDK version (`android:minSdkVersion`) is at least API level 11.

Android SDK version 11 is required to properly support input type constants such as `textWebPassword` and `textNoSuggestions`, which prevent the keyboard from caching sensitive user input. If `minSdkVersion` is set below 11, the compiled application may not honour these constants, allowing the keyboard to cache passwords and other sensitive data entered by the user, as described in MASTG-TEST-0006.

The test fails if `minSdkVersion` is below 11.

Test result: **Test failed** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **Test failed**

Evidence:

```
minSdkVersion is set to: None
```

47 References to Backup Configurations Not Excluding Sensitive Data

This test verifies whether the application prevents sensitive data from being included in Android backups.

When `android:allowBackup` is enabled (the default if the attribute is absent), Android may back up the application's data via ADB or cloud backup services. Without proper backup exclusion rules defined via `android:fullBackupContent` (Android 11 and below) or `android:dataExtractionRules` (Android 12 and above), sensitive data such as authentication tokens, credentials, or private files may be extracted from a backup.

The test fails if backups are enabled but no backup configuration with exclusion rules is defined.

Test result: **0.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.000**

Evidence:

```
android:allowBackup = false
Backup is disabled
```

72 The app is transparent about data collection and usage.

This test verifies that the application correctly declares dangerous permissions for accessing sensors and local user data.

Permissions that access sensitive resources such as location, camera, microphone, contacts, SMS, and body sensors must be declared with protection level "dangerous" so that Android prompts the user for explicit runtime consent. The test merges both app-defined and requested permissions from the manifest, then checks whether each relevant permission is effectively dangerous — either by explicit declaration or because Android treats it as dangerous by default.

The test fails if any sensor- or local-data-related permission is not effectively dangerous. It passes if all such permissions have appropriate protection levels, or if no relevant permissions are declared.

Test result: **0.000** Test weight: **MEDIUM RISK** Weight value: **0.062** Test outcome: **0.000**

80 The app is transparent about data collection and usage.

This test verifies whether the application provides a link to a privacy policy in the Google Play Store.

A privacy policy is a legal requirement in most jurisdictions for applications that collect or process user data. Its absence may indicate that the developer has not disclosed how user data is handled, leaving users without the information needed to make informed decisions about their privacy.

The test fails if no valid privacy policy URL is found in the Play Store listing.

Test result: **0.000** Test weight: **MEDIUM RISK** Weight value: **0.062** Test outcome: **0.000**

Evidence:

Privacy policy found: <https://sites.google.com/view/hideu-privacypolicy/>

90 Testing Enforced Updating

This test verifies whether the application implements the Google Play In-App Updates API.

Apps that lack an in-app update mechanism cannot prompt users to upgrade when security patches are available, leaving users exposed to known vulnerabilities in older versions. The test searches for `AppUpdateManager`, `AppUpdateManagerFactory`, `AppUpdateInfo`, and related API patterns in both Java and Smali source code.

The test passes if in-app update patterns are detected and fails if no update mechanism is found.

Test result: **0.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.000**

Evidence:

Found 141 inappupdate patterns

Note For brevity only 10 are shown here.

Found in-app update pattern 'AppUpdateType' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/jadx/sources/com/google/android/play/core/appupdate/testing/FakeAppUpdateManager.java

Found in-app update pattern 'AppUpdateInfo' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes7/com/amber/lib/update/gp/AllowedAvailableUpdateListener\$DefaultImpls.smali

Found in-app update pattern 'AppUpdateType' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/jadx/sources/com/google/android/play/core/install/model/AppUpdateType.java

Found in-app update pattern 'completeUpdate' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes7/com/amber/lib/update/gp/AllowedAvailableUpdateListener\$DefaultImpls.smali

Found in-app update pattern 'startUpdateFlowForResult' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes11/com/amber/lib/update/gp/UpdateManager.smali

Found in-app update pattern 'getAppUpdateInfo' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes11/com/amber/lib/update/gp/UpdateManager.smali

Found in-app update pattern 'AppUpdateManager' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/jadx/sources/com/calculator/hideu/setting/SettingActivity.java

Found in-app update pattern 'AppUpdateInfo' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/jadx/sources/com/google/android/play/core/appupdate/AppUpdateManager.java

Found in-app update pattern 'AppUpdateOptions' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes11/com/google/android/play/core/appupdate/testing/FakeAppUpdateManager.smali

Found in-app update pattern 'completeUpdate' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/jadx/sources/com/calculator/hideu/setting/SettingActivity.java

91 Testing Enforced Updating

This test validates whether the application enforces IMMEDIATE in-app updates that block continued use until the update is installed.

This test specifically checks whether updates are configured as IMMEDIATE (AppUpdateType=1) rather than FLEXIBLE (AppUpdateType=0). Flexible updates allow users to defer installing the update, leaving them on potentially vulnerable versions. The test analyses AppUpdateType assignments in Java and Smali bytecode.

The test passes if IMMEDIATE update enforcement is detected and fails if only FLEXIBLE updates are configured or no valid enforcement is found.

Test result: **1.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.021**

Evidence:

Found 145 in-app update patterns.

For Brevity only 10 are shown here:

Found in-app update pattern 'AppUpdateManager' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes6/com/calculator/hideu/setting/SettingActivity\$00o0000.smali

Found in-app update pattern 'appUpdateType=' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/jadx/sources/com/google/android/play/core/appupdate/zzx.java

Found in-app update pattern 'AppUpdateType' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes10/com/google/android/play/core/appupdate/AppUpdateOptions\$Builder.smali

Found in-app update pattern 'AppUpdateManager' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes6/com/google/android/play/core/appupdate/zzz.smali

Found in-app update pattern 'startUpdateFlowForResult' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/jadx/sources/com/google/android/play/core/appupdate/zzg.java

Found in-app update pattern 'AppUpdateInfo' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes12/ambercore/Ff0.smali

Found in-app update pattern 'AppUpdateManager' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/jadx/sources/com/amber/lib/update/gp/UpdateManager.java

Found in-app update pattern 'AppUpdateManager' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/jadx/sources/com/google/android/play/core/appupdate/testing/FakeAppUpdateManager.java

Found in-app update pattern 'AppUpdateOptions' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/jadx/sources/com/google/android/play/core/appupdate/zzg.java

Found in-app update pattern 'AppUpdateManager' in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes7/com/amber/lib/update/gp/AllowedAvailableUpdateListener\$DefaultImpls.smali

No valid update type configuration found

No valid update mode found in Smali files: ['tmp/

7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali/ambercore/Df0.smali']

96 Testing Local Storage for Input Validation

This test identifies instances where the application uses SharedPreferences to store sensitive data.

SharedPreferences stores data as plain XML files on the device filesystem and provides no encryption by default. Storing sensitive information such as passwords, tokens, or session identifiers in SharedPreferences exposes that data to other applications on rooted devices or during device backups. The test scans Java and Smali source files for getSharedPreferences() calls that occur near sensitive keywords.

The test fails if SharedPreferences usage is detected in a context that involves sensitive data.

Test result: **1.000** Test weight: **MEDIUM RISK** Weight value: **0.062** Test outcome: **0.062**

Evidence:

```
Shared preference usage found, at least in: {'tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/jadx/sources/com/amber/hideu/browser/ui/dialog/SniffVideoDialog.java', 'getSharedPreferences'}}
```

98 Determining Whether Sensitive Stored Data Has Been Exposed via IPC Mechanisms

This test verifies whether the application exposes sensitive data through improperly configured Content Providers.

Content Providers may be accessible to other applications when they are exported, either explicitly via android:exported="true" or implicitly through the presence of an <intent-filter>. In such cases, external applications may interact with the provider to read, insert, update, or delete data.

The test evaluates whether adequate access control mechanisms are implemented using:

```
android.permission
android.readPermission
android.writePermission
```

If a Content Provider is exported without appropriate access restrictions, this may lead to unauthorized access to or manipulation of potentially sensitive data.

Test result: **1.000** Test weight: **MEDIUM RISK** Weight value: **0.062** Test outcome: **0.062**

Evidence:

```
Provider 'androidx.core.content.FileProvider' is not exported
Provider 'androidx.startup.InitializationProvider' is not exported
Provider 'com.android.launcher3.LauncherProvider' is exported with access control
Provider 'com.facebook.internal.FacebookInitProvider' is not exported
Provider 'com.vungle.ads.VungleProvider' is not exported
Provider 'com.spirit.ads.utils.SdkContentProvider' is not exported
Provider 'com.amber.lib.billing.BillingProvider' is exported without access control
```

99 Determining Whether Sensitive Stored Data Has Been Exposed via IPC Mechanisms

This test verifies whether the application exposes sensitive information through Android Inter-Process Communication (IPC) mechanisms.

The script scans the application source code for IPC communication patterns such as Intents, Broadcasts, Services, and Content Providers. It then checks whether sensitive information may be transmitted through these IPC channels.

The goal is to detect cases where sensitive data (e.g., passwords, tokens, API keys, session identifiers) may be shared with other applications through IPC mechanisms.

The test fails if sensitive information is detected in combination with IPC communication patterns.

Test result: **1.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.021**

Evidence:

```
IPC communication patterns found, at least in: {'tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/jadx/sources/ambercore/AbstractC0922Ab0.java', 'registerReceiver()'}
```

100 Determining Whether Sensitive Stored Data Has Been Exposed via IPC Mechanisms

This test verifies whether custom Android permissions defined in the AndroidManifest.xml file are configured with the protection level signature.

Permissions with the value signature restrict access to applications signed with the same certificate. This mechanism is commonly used to protect IPC components such as Content Providers, Services, or Broadcast Receivers from unauthorized access.

The script analyzes the AndroidManifest.xml file and verifies whether the android:protectionLevel attribute is set to signature.

Test result: **1.000** Test weight: **MEDIUM RISK** Weight value: **0.062** Test outcome: **0.062**

Evidence:

```
FAIL: com.android.launcher.permission.INSTALL_SHORTCUT → protectionLevel=dangerous
FAIL: com.calculator.hideu.permission.READ_SETTINGS → protectionLevel=normal
FAIL: com.calculator.hideu.permission.WRITE_SETTINGS → protectionLevel=signatureOrSystem
OK: com.calculator.hideu.permission.READ_NOTIFICATIONS → protectionLevel=signature
FAIL: com.calculator.hideu.permission.UPDATE_COUNT → protectionLevel=normal
OK: com.calculator.hideu.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION → protectionLevel=signature
```

119 References to Screen Capturing Prevention APIs

This test verifies whether the application references Android APIs that prevent screenshots and screen recording.

On Android, developers can prevent screen capturing by setting the FLAG_SECURE flag on application windows. When this flag is enabled, the operating system blocks screenshots, screen recording, and nonsecure display mirroring.

Developers typically apply this protection using methods such as:

- addFlags(WindowManager.LayoutParams.FLAG_SECURE)
- setFlags(WindowManager.LayoutParams.FLAG_SECURE)

If the flag is not applied to screens displaying sensitive information, such as authentication pages or personal data views, sensitive data may be exposed through screenshots or screen sharing.

Test result: **0.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.000**

Evidence:

```
FLAG_SECURE is set at least in: {'tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/jadx/sources/androidx/compose/ui/window/DialogWrapper.java', 'setFlags(r33, 8192)'}
```

142 Testing Obfuscation

This test verifies whether the application uses code obfuscation techniques.

Obfuscation helps protect application logic and sensitive implementation details from reverse engineering. The test determines whether the APK contains indicators of obfuscation frameworks such as ProGuard, R8, DexGuard, or other protection mechanisms.

The implementation performs a static analysis of the APK using APKID to detect common obfuscation indicators.

Test result: **1.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.021**

Evidence:

No obfuscation detected.

159 References to Root Detection Mechanisms

This test verifies whether the application implements root detection techniques.

Applications often detect rooted devices to prevent execution in potentially compromised environments. Root detection mechanisms typically check for:

- the presence of root binaries
- root management applications
- execution of root commands
- root detection libraries

The implementation performs static analysis on the decompiled application code to detect indicators that root detection logic is present.

Test result: **0.000** Test weight: **MEDIUM RISK** Weight value: **0.062** Test outcome: **0.000**

Evidence:

```
Root detection mechanisms found, at least in: {'tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/jadx/sources/com/apm/insight/
nativecrash/a.java', '/system/xbin/su'), ('tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/jadx/sources/com/apm/insight/
nativecrash/a.java', '/sbin/su'), ('tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/jadx/sources/com/apm/insight/
nativecrash/a.java', '/system/bin/su')}
```

165 Checking for Weaknesses in Third Party Libraries

This test verifies whether the application includes third-party dependencies that contain known vulnerabilities.

Applications frequently rely on external libraries such as networking frameworks, JSON parsers, analytics SDKs, and other utility libraries. If these libraries contain known security vulnerabilities, the application may inherit those vulnerabilities.

The implementation performs static analysis of the decompiled application code to detect third-party libraries based on their package hierarchy. Detected libraries are then checked against a vulnerability database to determine whether known security issues exist.

Test result: **0.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.000**

Evidence:

Found 1906 third-party package(s):

Note for brevity only 20 are listed:

```
ambercore
arch.talent.permissions
cn.bingoogolapple.qrcode.core
cn.bingoogolapple.qrcode.zxing
coil.base
com.airbnb.lottie
com.amber.hideu.base.model
com.android.billingclient
com.android.billingclient.api
com.android.installreferrer
com.applovin.impl.adview
com.applovin.impl.adview.activity
com.applovin.impl.mediation.ads
com.applovin.impl.privacy.consentFlow
com.bykv.vk.openvk.gecko
com.bytedance
com.bytedance.adsdk.ZYk
com.bytedance.sdk.component.adexpress
com.bytedance.sdk.openadsdk.api.bidding
v3.arch.permissions
```

Filtering vulnerabilities published after version 2.3.8 (2026-03-19)

Loaded vulnerability index with 1093 unique Maven groupIds

Filtering vulnerabilities published after 2026-03-19

0/1906 packages matched at least one vulnerability

No known HIGH/CRITICAL vulnerabilities found for detected packages

166 The app is adhering the copyrights of the licenses.

This test evaluates whether applications properly comply with open-source license obligations when using third-party libraries.

Applications often include external libraries that are distributed under licenses such as Apache 2.0, MIT, BSD, or GPL. These licenses may require attribution, inclusion of license texts, or in the case of copyleft licenses, sharing of source code.

The analysis detects third-party libraries within the application and verifies whether required license files and disclosures are present. Missing license files or use of restrictive licenses without proper compliance may introduce legal and compliance risks.

Test result: **0.500** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.010**

Evidence:

Searching for license files...

Checking Play Store disclosures...

- x No mention of third-party libraries in description
- ✓ Privacy policy URL found: <https://sites.google.com/view/hideu-privacypolicy/>
- ✓ Developer website found: <https://amberweather.com/>

Detected library: Gson
Detected library: Jackson
Detected library: Retrofit
Detected library: Picasso
Detected library: Glide
Detected library: Firebase
Detected library: Facebook SDK
Detected library: Google Play Services
Detected library: SLF4J
Detected library: Jsoup
Detected library: Guava

Checking license requirements for detected libraries...

- ✓ Gson: Apache-2.0
- ✓ Jackson: Apache-2.0
- ✓ Retrofit: Apache-2.0
- ✓ Picasso: Apache-2.0
- ✓ Glide: BSD-2-Clause
- ✓ Firebase: Apache-2.0
- ✓ Facebook SDK: Apache-2.0
- ✓ Google Play Services: Apache-2.0
- ✓ SLF4J: MIT
- ✓ Jsoup: MIT
- ✓ Guava: Apache-2.0

LICENSE COMPLIANCE ASSESSMENT

- x No license files found in APK (+3 risk)
- x No third-party disclosure in app description (+2 risk)

TOTAL RISK SCORE: 5

 POOR COMPLIANCE - Significant issues found

Violations:

- No LICENSE/NOTICE files found in APK
- App description doesn't mention third-party libraries

Recommendations:

1. Add LICENSE/NOTICE file to app assets with third-party attributions
2. Update app description to mention use of third-party libraries

180 Android App Configurations Allowing Cleartext Traffic

This test verifies whether the application allows cleartext network traffic.

Allowing cleartext traffic means that network communication may occur over unencrypted protocols such as HTTP instead of HTTPS.

This can expose sensitive information to interception or manipulation.

Android applications can enable or disable cleartext traffic through:

- the android:usesCleartextTraffic attribute in AndroidManifest.xml
- a network_security_config.xml configuration file

The implementation currently performs static analysis on the manifest file to determine whether cleartext traffic is explicitly enabled.

Test result: **1.000** Test weight: **MEDIUM RISK** Weight value: **0.062** Test outcome: **0.062**

Evidence:

Found True in application.android:usesCleartextTraffic

187 Outdated Android Version Allowing Trust in User-Provided CAs

This test verifies whether the application trusts user-installed certificate authorities.

Applications targeting Android API level 23 or lower automatically trust user-installed certificates. This behavior allows attackers to install a malicious certificate authority on the device and intercept encrypted network traffic.

Applications targeting API level 24 or higher restrict this behavior by default unless the application explicitly allows it through a Network Security Configuration.

The current implementation performs static analysis on the application manifest to determine the application's target SDK version.

Test result: **1.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.021**

Evidence:

Target SDK version is higher than 23

User certificate trust found in tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/res/xml/hideu_network_security_config.xml

User certificate trust is defined in the Network Security Configuration

190 Testing Endpoint Identify Verification

This test evaluates whether the application is signed with a valid certificate by checking the certificate's validity period.

APK signing certificates contain a validity window defined by a start date ("not before") and an end date ("not after"). If the certificate is expired or not yet valid, the integrity and trustworthiness of the application may be compromised.

The analysis extracts certificate information using tooling and compares the validity period with the current time. Applications signed with invalid certificates may indicate improper signing practices or potential security risks.

Test result: **0.000** Test weight: **MEDIUM RISK** Weight value: **0.062** Test outcome: **0.000**

Evidence:

Output from androguard: com_calculator_hideu_v2.3.8.apk, package: 'com.calculator.hideu'

Is signed v1: True

Is signed v2: True

Is signed v3: True

Found 1 unique certificates

Issuer: O=Amber, OU=Mobile, CN=hideu

Subject: O=Amber, OU=Mobile, CN=hideu

Serial Number: 0x45ece00c

Hash Algorithm: sha256

Signature Algorithm: rsassa_pkcs1v15

Valid not before: 2021-04-30 15:21:08+00:00

Valid not after: 2046-04-24 15:21:08+00:00

sha1 7cfa48763617ab7d86d3e649ab40063cd2a0cddb

Found 1 unique public keys associated with the certs

Algorithm: rsa

Bit size: 2048

Fingerprint: 84677556635f541f10fb0681e28516e4c22e6741d637a54f8075d6a6d7d0520c

not_before: 2021-04-30 15:21:08+00:00

not_after: 2046-04-24 15:21:08+00:00

Certificate is valid

191 Usage of Insecure APK Signature Version

This test verifies whether the Android application uses a secure APK signing scheme.

APK Signature Scheme v1 signs individual files and may allow APK modification.

Newer schemes (v2, v3, v4) sign the entire APK and provide stronger integrity protection.

The test should therefore verify that the APK is not signed exclusively with v1.

Test result: **0.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.000**

Evidence:

Verifies

Verified using v1 scheme (JAR signing): true

Verified using v2 scheme (APK Signature Scheme v2): true

Verified using v3 scheme (APK Signature Scheme v3): true

Verified using v3.1 scheme (APK Signature Scheme v3.1): false

Verified using v4 scheme (APK Signature Scheme v4): false

Verified for SourceStamp: true

Number of signers: 1

Source Stamp Timestamp: 1773834506

Found 124 components not protected by signature.

For brevity only 10 are shown here.

WARNING: META-INF/CHANGES not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or move the entry outside of META-INF/.

WARNING: META-INF/README.md not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or move the entry outside of META-INF/.

WARNING: META-INF/androidx.activity_activity-compose.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or move the entry outside of META-INF/.

WARNING: META-INF/androidx.activity_activity-ktx.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or move the entry outside of META-INF/.

WARNING: META-INF/androidx.activity_activity.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or move the entry outside of META-INF/.

WARNING: META-INF/androidx.annotation_annotation-experimental.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or move the entry outside of META-INF/.

WARNING: META-INF/androidx.appcompat_appcompat-resources.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or move the entry outside of META-INF/.

WARNING: META-INF/androidx.appcompat_appcompat.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or move the entry outside of META-INF/.

WARNING: META-INF/androidx.arch.core_core-runtime.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or move the entry outside of META-INF/.

WARNING: META-INF/androidx.asynclayoutinflater_asynclayoutinflater.version not protected by signature. Unauthorized modifications to this JAR entry will not be detected. Delete or move the entry outside of META-INF/.

Signing version is 3.0

196 Debuggable Flag Enabled in the AndroidManifest

This test verifies whether the application has the debuggable flag enabled.

Apps with `android:debuggable="true"` can be debugged via ADB, which may expose the application to runtime analysis.

Production apps should not be debuggable.

Test result: **0.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.000**

Evidence:

Could not find ['True'] in `application.android:debuggable`

199 WebViews Not Cleaning Up Sensitive Data

This test assesses whether WebView cache is properly cleared from both memory and disk.

The method `WebView.clearCache(boolean includeDiskFiles)` is used to clear cached data. When called with true, both memory and disk cache are cleared. When called with false, only memory cache is cleared, and disk cache remains.

If disk cache is not cleared, sensitive data may persist on the device, even though cleanup appears to be performed.

The analysis verifies whether `clearCache(false)` is used, which indicates incomplete cleanup and potential data persistence risks.

Test result: **0.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.000**

Evidence:

Found 1 files with `clearCache` method

No `clearCache` method with `includeDiskFiles` set to false found

210 Testing for URL Loading in WebViews

This test checks whether the application overrides WebView's default URL loading and request interception behavior via `WebViewClient` callback methods.

The `shouldOverrideUrlLoading` and `shouldInterceptRequest` callbacks allow an application to intercept and inspect web navigation and network requests made inside a WebView. Their presence indicates that the application has custom logic for handling web traffic, which may introduce security issues if the implementation is incorrect or incomplete.

The test fails if either `shouldOverrideUrlLoading` or `shouldInterceptRequest` is detected in the decompiled source code.

Test result: **1.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.021**

Evidence:

Page navigation handling override found in: `{('tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali/com/bytedance/sdk/component/jfA/ba$0J.smali', 'shouldInterceptRequest'), ('tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali/com/bytedance/sdk/component/jfA/ba$0J.smali', 'shouldOverrideUrlLoading')}`

212 Testing for URL Loading in WebViews

This test verifies whether the application has explicitly disabled WebView's Safe Browsing protection.

Safe Browsing is enabled by default in Android WebView and warns users before loading known phishing or malware sites. Disabling it by setting `android.webkit.WebView.EnableSafeBrowsing` to false in the manifest removes this protection and exposes users to malicious web content loaded inside the application's WebView.

The test fails if the `EnableSafeBrowsing` meta-data attribute is explicitly set to false in the `AndroidManifest.xml` file.

Test result: **0.000** Test weight: **MEDIUM RISK** Weight value: **0.062** Test outcome: **0.000**

Evidence:

Could not find ['False'] in `meta-data.android.webkit.WebView.EnableSafeBrowsing`

221 The Developer is has mainly good reviews

This test verifies whether the application's average user review score on the Google Play Store is above 3.0.

A consistently low average rating may indicate poor application quality, frequent crashes, or significant user dissatisfaction, which can indirectly reflect on the overall reliability and trustworthiness of the application and its developer.

The test passes if the average score is above 3. If the score is 3 or lower, or if no score data is available, the test fails.

Test result: **0.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.000**

Evidence:

Play Store score: 4.74

222 The Developer is has mainly good reviews

This test analyzes user reviews in the Google Play Store to detect recurring negative patterns that may indicate security or reliability concerns.

Reviews are examined for red flags such as crash reports, privacy complaints, misleading behavior, hidden costs, or poor developer support. A significant proportion of such reviews may signal that the application poses a risk to users or is not well-maintained.

The test fails if the proportion of reviews containing red flags exceeds an acceptable threshold.

Test result: **0.000** Test weight: **MEDIUM RISK** Weight value: **0.062** Test outcome: **0.000**

223 The developer complies with Google standards and discloses all information in the playstore (use of data etc) Failing to do so could hide malicious intent

This test verifies whether the application developer has provided data handling information in the Google Play Store's Data Safety section.

App store policies and privacy regulations require developers to disclose how their applications collect, share, and handle user data. The absence of this transparency statement may indicate that the developer has not completed their data safety disclosure, which is a concern for user privacy and regulatory compliance.

The test fails if the developer's data handling statement is not found on the Play Store page.

Test result: **0.000** Test weight: **MODERATE ISSUE** Weight value: **0.300** Test outcome: **0.000**

Evidence:

Found developer information in <https://play.google.com/store/apps/datasafety?id=com.calculator.hideu&hl=nl>

226 The app is regularly updated

This test verifies whether the application has been updated within the past three months.

Actively maintained applications receive regular security patches and bug fixes. An application that has not been updated in more than 90 days may rely on outdated or vulnerable dependencies, or may have been abandoned by its developer, leaving known security issues unaddressed.

The test fails if the last update timestamp in the Play Store data indicates more than 90 days have passed since the most recent release.

Test result: **0.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.000**

Evidence:

Last update timestamp: 1773834534

227 The app is regularly updated

This test verifies whether the application has received at least four updates in the past twelve months, indicating active maintenance.

Actively maintained applications are more likely to receive timely security patches and bug fixes. An application with fewer than four updates in a rolling twelve-month window may be under-maintained, leaving known vulnerabilities unaddressed for extended periods. Version history is read from the Play Store scraper data cached during the analysis run.

The test fails if fewer than four releases are found within the past twelve months.

Test result: **0.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.000**

Evidence:

Latest version dates: ['Apr 11, 2024', 'Apr 22, 2024', 'Aug 15, 2025', 'Aug 20, 2025', 'Aug 24, 2023', 'Aug 24, 2024', 'Aug 26, 2023', 'Aug 9, 2023', 'Aug 9, 2023', 'Dec 13, 2023', 'Dec 15, 2023', 'Dec 21, 2023', 'Dec 8, 2023', 'Feb 14, 2023', 'Jan 10, 2024', 'Jan 8, 2024', 'Jul 26, 2023', 'Jul 26, 2023', 'Jul 5, 2023', 'Mar 17, 2026', 'Mar 19, 2026', 'May 31, 2023', 'Nov 15, 2023', 'Nov 2, 2023', 'Nov 28, 2023', 'Oct 10, 2023', 'Oct 24, 2023', 'Oct 26, 2023', 'Oct 30, 2025', 'Sep 14, 2023', 'Sep 21, 2023', 'Sep 5, 2023']

228 The app is used by a large group of users

This test verifies whether the application has reached at least 100,000 real installs on the Google Play Store.

A low install count may indicate a relatively unknown application that has not undergone broad community scrutiny. Widely-used applications are generally more likely to have security issues discovered and reported, while lesser-known apps may carry undetected risks and have not been validated by a large user base.

The test fails if the reported real installation count is below 100,000.

Test result: **0.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.000**

Evidence:

Real installations: 249904374

230 Testing Deep Links

This test checks whether the application defines deep links in its AndroidManifest.xml file.

Deep links using custom URL schemes or verified app links allow external applications and web pages to trigger specific activities within the app. Incorrectly validated deep link handlers can be exploited to bypass authentication flows, pass untrusted data into the application, or leak sensitive information to malicious callers. The test inspects intent filters for the presence of BROWSABLE or DEFAULT categories combined with scheme and host data elements.

The test fails if deep link definitions are found in the manifest.

Test result: **1.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.021**

Evidence:

Extracted grouped component deeplinks: defaultdict(<class 'list'>,
{'com.calculator.hideu.calculator2.ui.retrieve.RetrieveCheckActivity': ['hideu://hideu.page.link']})

231 Testing Deep Links

This test verifies whether deep links configured with android:autoVerify="true" are properly associated with their claimed domains via the Digital Asset Links protocol.

Android App Links require the app's package name and signing certificate to be listed in a /.well-known/assetlinks.json file hosted on the associated domain. If this file is missing or unreachable, the link cannot be verified as an App Link, and other applications may intercept the intent, enabling phishing attacks or unintended data exposure.

The test fails if a domain is claimed with autoVerify="true" but the corresponding assetlinks.json file is not reachable or returns a non-200 response.

Test result: **1.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.021**

234 Testing for Sensitive Functionality Exposure Through IPC

This test checks whether Android activities are exposed (explicitly or implicitly) without proper permission protection, in alignment with OWASP MASTG-TEST-0029.

An activity is considered exposed if:

- android:exported="true" (explicitly exposed)
- <intent-filter> is present (implicitly exposed)

Exposed activities without android:permission protection may allow unauthorized IPC access from other applications.

Launcher activities are excluded as they are expected to be exported.

Test result: **1.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.021**

Evidence:

Risky activity: com.calculator.hideu.calculator2.view.TestActivity
Risky activity: com.calculator.hideu.calculator2.ui.retrieve.RetrieveCheckActivity
Risky activity: com.anddoes.launcher.Launcher
Risky activity: com.anddoes.launcher.settings.ui.ShortcutActivity
Risky activity: com.facebook.CustomTabActivity
Found 5 risky activities

235 Testing for Overlay Attacks

This test checks whether the application implements protections against overlay attacks, where a malicious app draws a transparent layer over the legitimate app's UI to intercept user input.

Overlay attacks can be used to capture credentials, trick users into granting permissions, or perform clickjacking. Protective mechanisms include setFilterTouchesWhenObscured, onFilterTouchEventForSecurity, TYPE_APPLICATION_OVERLAY, TYPE_SYSTEM_ALERT, and the FLAG_WINDOW_IS_OBSCURED flags. The presence of these patterns indicates that the app is aware of and actively addressing overlay threats.

The test fails if none of the expected overlay protection mechanisms are detected in the decompiled source code.

Test result: **1.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.021**

Evidence:

Possible overlay attack pattern found, at least in: {'tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes10/com/bytedance/sdk/openadsdk/core/jFA/oJ/ZYk.smali', 'SYSTEM_ALERT_WINDOW'}}

243 Ensure that financial transaction are visible to the user and with consent

This test evaluates whether applications correctly implement Google Play Billing when offering digital content.

Digital content includes:

- Premium features
- Subscriptions
- Digital unlocks
- In-app currencies or credits

The presence of in-app purchases (IAP) is used as the primary indicator that the app provides digital goods.

According to Google Play policy, all digital goods and services must be purchased using Google Play Billing.

The analysis verifies:

- Whether IAP is present
- Whether Google Play Billing is implemented

Test result: **0.000** Test weight: **MODERATE ISSUE** Weight value: **0.300** Test outcome: **0.000**

Evidence:

Payment option found in playstore data: True
Found com.android.vending.BILLING in uses-permission.android:name
Billing permission found in manifest file
IAP and billing permission both present – compliant

244 When facilitating financial transactions, Validate that the app uses a well known payment platform

This test evaluates whether an application provides digital goods and ensures correct billing behavior.

Digital content includes:

- Premium features
- Subscriptions
- Digital unlocks
- In-app currencies or credits

According to Google Play policy:

- Digital content must use Google Play Billing
- External payment methods are not allowed for digital goods

The test correlates:

- Play Store metadata (IAP)
- Billing implementation (permission + flow)
- Payment SDK usage
- Digital content indicators (code/UI)
- Physical commerce indicators (to distinguish allowed flows)

This ensures context-aware validation instead of relying on payment method recognition alone.

Test result: **0.000** Test weight: **HIGH RISK** Weight value: **0.123** Test outcome: **0.000**

Evidence:

External payment method found in tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8: {('tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes11/com/google/common/util/concurrent/0oo00o0.smali', 'Stripe')}

Physical commerce indicator 'carrier' found in tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes6/com/applovin/impl/sdk/l.smali

Physical commerce indicator 'shipping' found in tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes9/com/google/firebase/analytics/FirebaseAnalytics\$Param.smali

Physical commerce indicator 'delivery' found in tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes2/com/fyber/inneractive/sdk/player/n.smali

Physical commerce indicator 'SKU' found in tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes7/com/android/billingclient/api/BillingFlowParams\$SubscriptionUpdateParams\$Builder.smali

Pass: No prohibited external payments detected

245 When facilitating financial transactions, Validate that the app uses a well known payment platform

This test evaluates whether applications that sell physical goods use a known external payment provider.

If physical commerce indicators are detected (shipping, delivery, warehouse, etc.) but no recognised external payment provider is found, the app may be handling payments in an unverified or non-transparent way.

The test fails when:

- Physical commerce is detected (2+ distinct keywords)
- No known external payment provider is found in the code

Test result: **0.000** Test weight: **HIGH RISK** Weight value: **0.123** Test outcome: **0.000**

Evidence:

Physical commerce indicator 'SKU' found in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes6/com/android/
billingclient/api/0000000.smali
Physical commerce indicator 'delivery' found in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes2/com/applovin/
impl/z7.smali
Physical commerce indicator 'carrier' found in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes11/com/facebook/
biddingkit/chartboost/ChartboostBidderPayloadBuilder.smali
Physical commerce indicator 'shipping' found in tmp/
7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes3/com/google/
android/gms/measurement/internal/zzjz.smali
External payment method found in tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8:
{('tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes11/com/
google/common/util/concurrent/0000000.smali', 'Stripe')}
Pass: Physical commerce detected with a known external payment provider

246 When facilitating financial transactions, make sure the app uses the proper (security) procedures

This test verifies whether applications that handle sensitive data implement proper authentication mechanisms (biometrics, PIN, password).

Authentication is only required for apps in sensitive categories such as financial services (banking, payments) and communication (messaging, email). Apps outside these categories pass regardless of authentication presence.

Test result: **0.000** Test weight: **HIGH RISK** Weight value: **0.123** Test outcome: **0.000**

Evidence:

App genre 'Tools' is not a sensitive category, skipped

247 When facilitating financial transactions, make sure the app uses the proper (security) procedures

This test verifies whether applications implementing authentication also enforce a lockout mechanism after repeated failed attempts.

Authentication mechanisms such as passwords, PINs, or biometrics should be complemented by a lockout mechanism that limits repeated failed attempts (e.g., maximum attempts, temporary lock, or account blocking).

The test first detects the presence of authentication-related APIs. If authentication is present, it then checks for lockout indicators. If no authentication is detected, the test is marked as Not Applicable.

Test result: **0.000** Test weight: **MEDIUM RISK** Weight value: **0.062** Test outcome: **0.000**

Evidence:

Authenticate detection mechanisms found, at least in: {'tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali/androidx/biometric/CryptoObjectUtils\$Api28Impl.smali', 'BiometricPrompt'}}
Lockout mechanisms found, at least in: {'tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali/androidx/biometric/R\$string.smali', 'lockout'}}
Lockout mechanism detected

248 Ensure that financial transactions are visible to the user and with consent

This test checks whether applications rated PEGI 3 or targeted at young children implement a parental consent mechanism before processing in-app purchases.

Children's applications that offer in-app purchases are required by app store policies and consumer protection regulations to obtain approval from a parent or guardian before completing a transaction. The test combines the Play Store content rating with the presence of payment functionality and scans the source code for parental consent patterns such as PARENTAL_CONSENT, PARENTAL_CONTROL, and PARENTAL_APPROVAL.

The test fails if the app is rated PEGI 3 (or has an equivalent child-targeted rating), offers in-app purchases, and no parental consent mechanism is detected in the source code.

Test result: **0.000** Test weight: **MEDIUM RISK** Weight value: **0.062** Test outcome: **0.000**

Evidence:

Content rating: PEGI 3
Payment option found in playstore data: True
Parental consent mechanism found, at least in: {'tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes8/android/support/v4/media/session/PlaybackStateCompat.smali', 'PARENTAL_CONTROL'}}

249 When facilitating financial transactions, make sure the app uses the proper (security) procedures

This test checks whether the application implements a mechanism to remotely disable or deactivate it in the event the device is lost or stolen.

Financial and enterprise applications should allow administrators or users to remotely revoke access, wipe session data, or deactivate the application to prevent unauthorized use. The test scans the source code for remote wipe and device administration patterns such as remote_wipe, disableApp, and DEVICE_ADMIN_.

The test fails if no remote disabling mechanism is detected in applications that offer payment functionality.

Test result: **0.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.000**

Evidence:

Payment option found in playstore data: True
Remote disabling mechanism found, at least in: {'tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes12/com/anddoes/launcher/ApexDeviceAdminReceiver.smali', 'DEVICE_ADMIN_'}}

252 When facilitating financial transactions, make sure the app uses the proper (security) procedures

SIM swap attacks allow attackers to take control of a user's phone number and intercept SMS-based authentication.

Applications may mitigate this risk by incorporating SIM or telephony information into their security model.

However, static analysis cannot confirm whether SIM changes are actively detected or enforced.

Therefore, this test only verifies whether the application uses SIM/telephony-related APIs.

Absence of such usage may indicate that the application does not use SIM-based risk signals.

Test result: **0.000** Test weight: **HIGH RISK** Weight value: **0.123** Test outcome: **0.000**

Evidence:

App genre 'Tools' is not in scope for SIM-awareness, skipped

255 Ensure that financial transaction are visible to the user and with consent

This test checks whether the application requests both in-app billing and SMS read permissions simultaneously.

The combination of the `com.android.vending.BILLING` permission with `READ_SMS` or `RECEIVE_SMS` is a pattern commonly associated with malicious or suspicious applications that may silently intercept SMS-based payment verification codes (OTPs) to facilitate unauthorized transactions. Legitimate payment applications rarely require both capabilities.

The test fails if the application declares both a billing permission and an SMS read or receive permission in its manifest.

Test result: **0.000** Test weight: **MEDIUM RISK** Weight value: **0.062** Test outcome: **0.000**

Evidence:

Could not find ['`android.permission.RECEIVE_SMS`', '`android.permission.READ_SMS`'] in `uses-permission.android:name`

Found `com.android.vending.BILLING` in `uses-permission.android:name`

257 Ensure that financial transaction are visible to the user and with consent

This test checks whether the application registers an Android Accessibility Service.

Accessibility Services have elevated privileges to observe all UI content and intercept user input across the entire device, including reading text from password fields. While this capability is intended for assistive technologies, it is frequently abused by banking trojans and malware to capture credentials, bypass authentication, or perform unauthorized actions on behalf of the user.

The test fails if the application registers a service with the `BIND_ACCESSIBILITY_SERVICE` permission in its manifest.

Test result: **0.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.000**

Evidence:

Could not find ['`android.permission.BIND_ACCESSIBILITY_SERVICE`'] in `service.android:permission`

258 References to APIs for Detecting Secure Screen Lock

This test verifies whether financial and communication applications detect the device's security state using Android platform APIs.

Applications handling sensitive data should verify that the device is protected by a secure lock mechanism (e.g., PIN, password, pattern, or biometrics) before allowing access or performing sensitive operations.

The analysis detects references to:

- KeyguardManager.isDeviceSecure()
- KeyguardManager.isKeyguardSecure()
- BiometricManager.canAuthenticate()

Absence of such checks may indicate increased risk of unauthorized access on unsecured devices.

Test result: **0.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.000**

Evidence:

App genre 'Tools' is not a sensitive category, skipped

262 The app does not collect or share too much information with 3rd parties

This test evaluates the privacy risk associated with data collected by the application, based on the Play Store's Data Safety section.

Each type of data collected (such as location, contacts, financial information, or health data) is assigned a sensitivity score, which is then multiplied by a weight reflecting the purpose for which it is collected (such as app functionality, analytics, or advertising). The combined score is normalized using a sigmoid function to produce a 0-1 risk value.

Higher scores indicate a broader or more sensitive data collection footprint, representing a greater privacy risk for users.

Test result: **0.071** Test weight: **MODERATE ISSUE** Weight value: **0.300** Test outcome: **0.021**

Evidence:

- Collected data: data=Device or other IDs, data_type=Device or other IDs, data_purpose=Analytics, Advertising or marketing, Fraud prevention, security, and compliance
- Collected data: data=App interactions, data_type=App activity, data_purpose=Analytics, Advertising or marketing
- Collected data: data=Installed apps, data_type=App activity, data_purpose=App functionality
- Collected data: data=Other actions, data_type=App activity, data_purpose=Analytics, Advertising or marketing
- Collected data: data=Crash logs, data_type=App info and performance, data_purpose=Analytics
- Collected data: data=Diagnostics, data_type=App info and performance, data_purpose=Analytics
- Collected data: data=Other app performance data, data_type=App info and performance, data_purpose=Analytics
- Collected data: data=User IDs, data_type=Personal info, data_purpose=Analytics

Data collection score: 125

Normalized score: 0.07126129851479564

263 The app does not collect or share too much information with 3rd parties

This test evaluates the privacy risk associated with data shared by the application with third parties, based on the Play Store's Data Safety section.

Each type of data shared (such as location, messages, financial information, or audio) is assigned a sensitivity score, which is multiplied by a weight reflecting the purpose for which it is shared (such as analytics, advertising, or personalization). The combined score is normalized using a sigmoid function to produce a 0-1 risk value.

Higher scores indicate that more sensitive data is shared for higher-risk purposes, representing a greater privacy concern for users.

Test result: **0.067** Test weight: **IMPACTFUL ISSUE** Weight value: **0.500** Test outcome: **0.034**

Evidence:

- Shared data: data=Device or other IDs, data_type=Device or other IDs, data_purpose=Analytics, Advertising or marketing, Fraud prevention, security, and compliance
- Shared data: data=App interactions, data_type=App activity, data_purpose=Analytics, Advertising or marketing
- Shared data: data=Installed apps, data_type=App activity, data_purpose=App functionality
- Shared data: data=Other actions, data_type=App activity, data_purpose=Analytics, Advertising or marketing
- Shared data: data=Crash logs, data_type=App info and performance, data_purpose=Analytics
- Shared data: data=Diagnostics, data_type=App info and performance, data_purpose=Analytics
- Shared data: data=Other app performance data, data_type=App info and performance, data_purpose=Analytics
- Shared data: data=User IDs, data_type=Personal info, data_purpose=Analytics

Data sharing score: 155

Normalized score: 0.0673673357863443

264 Stack Canaries Not Enabled

This test verifies whether the native libraries (.so files) in the application are compiled with stack smashing protection (stack canaries).

Stack canaries are a compile-time mitigation that places a known sentinel value on the stack before the return address. If a buffer overflow overwrites the canary, the runtime detects the corruption and terminates the process instead of executing attacker-controlled code. Native libraries compiled without this protection are significantly more vulnerable to memory corruption exploits.

The test fails if any native library is found without stack canary protection enabled.

Test result: **1.000** Test weight: **HIGH RISK** Weight value: **0.123** Test outcome: **0.123**

Evidence:

Checking native libs for canary...

Found no canary protection in tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/lib/arm64-v8a/libcrashlytics-trampoline.so

265 Position Independent Code (PIC) Not Enabled

This test verifies whether the native libraries (.so files) in the application are compiled with Position Independent Code (PIC).

PIC, combined with the operating system's Address Space Layout Randomization (ASLR), ensures that code is loaded at a randomized address on each execution. This makes it significantly harder for attackers to predict memory locations and construct exploits such as return-oriented programming (ROP) chains. Native libraries compiled without PIC are loaded at fixed addresses, removing this protection.

The test fails if any native library is found without PIC enabled.

Test result: **0.000** Test weight: **HIGH RISK** Weight value: **0.123** Test outcome: **0.000**

Evidence:

Checking native libs for pic...

All scanned lib files have pic protection in tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8

266 The app does not collect or share too much information with 3rd parties

This test evaluates the overall permission footprint of the application by analysing all defined and requested permissions in the AndroidManifest.xml file.

Each permission is classified as either dangerous or normal based on Android's permission model. Dangerous permissions carry a higher weight in the scoring model because they grant access to sensitive data or device capabilities. A scaled sigmoid function normalizes the combined score to the 0-1 range, where higher scores indicate a greater privacy and security risk from the app's permission requests.

The test produces a continuous risk score rather than a binary pass/fail result.

Test result: **1.000** Test weight: **HIGH RISK** Weight value: **0.123** Test outcome: **0.123**

Evidence:

```
Android permission : android.permission.EXPAND_STATUS_BAR
  Android permission : com.calculator.hideu.permission.READ_SETTINGS
  Android permission : com.calculator.hideu.permission.READ_NOTIFICATIONS
  Android permission : com.hihonor.calendar.permission.WRITE_CALENDAR_RECESS
  Android permission : com.google.android.finsky.permission.BIND_GET_INSTALL_REFERRER_SERVICE
  Android permission : android.permission.SYSTEM_ALERT_WINDOW
  Android permission : android.permission.CHANGE_COMPONENT_ENABLED_STATE
!! Dangerous permission : android.permission.ACCESS_FINE_LOCATION
!! Dangerous permission : android.permission.WRITE_EXTERNAL_STORAGE
!! Dangerous permission : android.permission.BLUETOOTH_CONNECT
  Android permission : com.android.vending.BILLING
  Android permission : android.permission.WAKE_LOCK
  Android permission : android.permission.USE_BIOMETRIC
  Android permission : com.calculator.hideu.permission.READ_NOTIFICATIONS
!! Dangerous permission : android.permission.ACCESS_COARSE_LOCATION
  Android permission : android.permission.WRITE_SETTINGS
  Android permission : com.applovin.array.apphub.permission.BIND_APPHUB_SERVICE
  Android permission : android.permission.SET_WALLPAPER
  Android permission : com.calculator.hideu.permission.UPDATE_COUNT
  Android permission : android.permission.VIBRATE
!! Dangerous permission : android.permission.RECORD_AUDIO
  Android permission : android.permission-group.com.calculator.hideu.SYSTEM_TOOLS
  Android permission : android.permission.FLASHLIGHT
  Android permission : android.permission.GET_PACKAGE_SIZE
  Android permission : com.htc.launcher.permission.READ_SETTINGS
  Android permission : com.calculator.hideu.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION
  Android permission : android.permission.BLUETOOTH
!! Dangerous permission : android.permission.NEARBY_WIFI_DEVICES
!! Dangerous permission : android.permission.READ_PHONE_STATE
!! Dangerous permission : android.permission.BLUETOOTH_SCAN
  Android permission : org.adw.launcher.permission.READ_SETTINGS
!! Dangerous permission : android.permission.BLUETOOTH_ADVERTISE
  Android permission : oppo.permission.OPPO_COMPONENT_SAFE
  Android permission : android.permission.ACCESS_WEATHERCLOCK_PROVIDER
  Android permission : com.calculator.hideu.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION
  Android permission : com.google.android.gms.permission.AD_ID
  Android permission : android.permission.INTERNET
  Android permission : com.cyanogenmod.trebuchet.permission.READ_SETTINGS
  Android permission : android.permission.ACCESS_WIFI_STATE
  Android permission : android.permission.ACCESS_AD SERVICES TOPICS
  Android permission : com.android.launcher.permission.READ_SETTINGS
  Android permission : android.permission.MANAGE_EXTERNAL_STORAGE
  Android permission : android.permission.ACCESS_AD SERVICES ATTRIBUTION
  Android permission : android.permission.USE_FINGERPRINT
  Android permission : android.permission.ACCESS_NETWORK_STATE
  Android permission : android.permission.REQUEST_DELETE_PACKAGES
!! Dangerous permission : android.permission.CAMERA
  Android permission : android.permission.REORDER_TASKS
  Android permission : android.permission.CHANGE_NETWORK_STATE
  Android permission : android.permission.ACCESS_AD SERVICES AD_ID
  Android permission : android.permission.BLUETOOTH_ADMIN
!! Dangerous permission : android.permission.READ_EXTERNAL_STORAGE
  Android permission : android.permission.FOREGROUND_SERVICE
```

```

Android permission : com.google.android.c2dm.permission.RECEIVE
Android permission : android.permission.ACCESS_SUPERUSER
Android permission : android.permission.CHANGE_WIFI_STATE
Android permission : com.android.launcher.permission.INSTALL_SHORTCUT
Android permission : com.android.launcher3.permission.READ_SETTINGS
Android permission : com.google.android.apps.nexuslauncher.permission.READ_SETTINGS
Android permission : com.hihonor.android.totemweather.permission.ACCESS_WEATHERCLOCK_PROVIDER
Android permission : android.permission.SET_WALLPAPER_HINTS
Android permission : android.permission.PACKAGE_USAGE_STATS
!! Dangerous permission : android.permission.READ_CONTACTS
Android permission : android.permission.FOREGROUND_SERVICE_SPECIAL_USE
Android permission : com.fede.launcher.permission.READ_SETTINGS
Android permission : android.permission.BIND_NOTIFICATION_LISTENER_SERVICE
Android permission : org.adwfreak.launcher.permission.READ_SETTINGS
Android permission : com.teslacoilsw.launcher.permission.READ_SETTINGS
Android permission : com.calculator.hideu.permission.WRITE_SETTINGS
Android permission : com.calculator.hideu.permission.READ_SETTINGS
Android permission : com.calculator.hideu.permission.WRITE_SETTINGS
Android permission : com.samsung.android.launcher.permission.READ_SETTINGS
!! Dangerous permission : android.permission.POST_NOTIFICATIONS
!! Dangerous permission : android.permission.CALL_PHONE
Android permission : com.android.calendar.hihonor.permission.CALENDAR_RECESS

```

267 The app should not use trackers.

This test analyses the application for the presence of known third-party tracking libraries using the Exodus Privacy tracker database.

Trackers embedded in applications collect user data for purposes such as analytics, advertising, or behavioral profiling, often without adequate disclosure or explicit user consent. The test uses class signature patterns from the Exodus Privacy project to identify known tracker SDKs in the decompiled source code.

The score ranges from 0 (no trackers) to 1 (ten or more trackers detected), scaled linearly.

Test result: **1.000** Test weight: **IMPACTFUL ISSUE** Weight value: **0.500** Test outcome: **0.500**

Evidence:

Found 15 trackers:

- Google CrashLytics
- Google Firebase Analytics
- Facebook Ads
- Facebook Analytics
- Facebook Login
- Facebook Share
- AppLovin (MAX and SparkLabs)
- Fyber
- Inmobi
- Unity3d Ads
- ironSource
- Mintegral
- Google AdMob
- IAB Open Measurement
- Pangle

268 The app should be listed in the Google Play Store.

This test verifies whether the application is available in the Google Play Store.

Apps distributed outside the official Play Store may not have passed Google's security review process and could be unofficial, tampered, or potentially malicious versions of the original application. The test checks whether the app's package name resolves to a valid listing on the Play Store.

The test fails if the application cannot be found in the Google Play Store.

Test result: **0.000** Test weight: **IMPACTFUL ISSUE** Weight value: **0.500** Test outcome: **0.000**

Evidence:

Checking if 'com.calculator.hideu' exists in Play Store...

270 WebViews Not Cleaning Up Sensitive Data

WebView DOM storage allows applications to store persistent data locally. If enabled, this data may remain on the device and expose sensitive information.

This test evaluates whether DOM storage is enabled and whether cleanup mechanisms such as `WebStorage.deleteAllData()` are implemented.

Test result: **1.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.021**

Evidence:

Method `setDomStorageEnabled` in `tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes7/com/bytedance/sdk/openadsdk/activity/TTWebsiteActivity.smali` has parameter set to `True`

Persistent dom is enabled but no files found with persistent dom cleanup

271 WebViews Not Cleaning Up Sensitive Data

WebView database storage allows persistent data storage on the device. If not properly cleared, sensitive data may remain accessible.

This test evaluates whether database storage is enabled and whether cleanup mechanisms such as `WebStorage.deleteAllData()` are implemented.

Test result: **1.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.021**

Evidence:

Method `setDatabaseEnabled` in `tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes11/com/mbridge/msdk/mbsignalcommon/base/BaseWebView.smali` has parameter set to `True`

Database storage is enabled but no files found with database storage cleanup

272 WebViews Not Cleaning Up Sensitive Data

WebView cache may store sensitive data locally. If cache is enabled without proper cleanup, this data may persist on the device.

This test evaluates whether cache is enabled and whether it is properly cleared using `WebView.clearCache(true)`.

Test result: **0.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.000**

Evidence:

Method `setCacheMode` in `tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes3/com/unity3d/services/core/webview/WebViewWithCache.smali` has parameter `-1` (not `2`)

Method `clearCache` found in `tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes10/com/inmobi/media/cf.smali`

273 WebViews Not Cleaning Up Sensitive Data

Cookies may store session or sensitive user data. If cookies are enabled and not properly cleared, they may persist and expose sensitive information.

This test evaluates whether cookies are enabled and whether cleanup mechanisms such as `CookieManager.removeAllCookies(...)` are implemented.

Test result: **0.000** Test weight: **LOW RISK** Weight value: **0.021** Test outcome: **0.000**

Evidence:

Method `removeAllCookies` found in `tmp/7ded060924019ac43583cd927b1870709e0caab3557f99bdeaeed66f2fef1ea8/apktool/smali_classes9/com/calculator/hideu/browser/work/ClearCookieWork$00o0000.smali`

274 When facilitating financial transactions, Validate that the app uses a well known payment platform

This test evaluates whether the application shows signs of payment-related problems by analysing user reviews from the Google Play Store.

Reviews are scanned for signals such as double charges, unexpected fees, billing errors, and refund denials in both Dutch and English. A significant proportion of payment complaints may indicate incorrect payment handling, a poor user experience, or potential financial risk for users of the application.

The test fails if payment complaints are found in more than 2% of reviews from apps that implement payment functionality.

Test result: **0.000** Test weight: **HIGH RISK** Weight value: **0.123** Test outcome: **0.000**

Evidence:

Payment option found in playstore data: True
Only 0 reviews available (< 10), inconclusive